Object-Based Architecture: Complete Guide with Examples

What is Object-Based Architecture?

Object-based architecture is a distributed computing model where objects (containing both data and methods) are distributed across different machines in a network. The client can invoke methods on remote objects as if they were local objects.

Key Components Explained

1. Client Machine

The machine where the user application runs and requests services.

Example: Your laptop running a banking application that needs to check your account balance.

2. Server Machine

The machine that hosts the actual business objects and processes requests.

Example: Bank's server containing customer account objects with balance data and transaction methods.

3. Object

A software component that encapsulates both data (state) and behavior (methods).

Example: A `BankAccount` object containing:

- **State:** account number, balance, customer name
- **Methods:** `deposit()`, `withdraw()`, `getBalance()`

4. State

The data/attributes stored within an object.

Example: In a `BankAccount` object:

```
accountNumber: "123456789"
balance: $5,000
customerName: "John Doe"
overdraftLimit: $500
```

5. Method

Functions that operate on the object's data.

Example: BankAccount methods:

```
deposit(amount) - adds money to balance
withdraw(amount) - removes money from balance
getBalance() - returns current balance
transferTo(targetAccount, amount) - moves money between accounts
```

6. Interface

The contract that defines what methods are available to clients.

Example: `IBankAccount` interface:

```
interface IBankAccount {
    void deposit(double amount);
    boolean withdraw(double amount);
    double getBalance();
    void transferTo(String targetAccount, double amount);
}
```

7. Proxy

A local representative of the remote object that handles communication details.

Example: When your banking app calls `account.getBalance()`, the proxy:

- Converts the method call into a network message
- Sends it to the server
- Receives the response
- Returns the balance to your app

The proxy makes the remote call appear local to your application.

8. Skeleton

The server-side component that receives network requests and invokes actual object methods.

Example: When the server receives a network request for `getBalance()`, the skeleton:

- Unmarshals the request
- Calls the actual `BankAccount.getBalance()` method
- Marshals the result back to send over network

How It All Works Together

Real-World Example: Online Shopping

1. **Client Side:**
   - You're using Amazon's mobile app (Client)
   - App has proxy objects for `ShoppingCart`, `Product`, `Order`
2. **Method Invocation:**
   - You tap "Add to Cart"
   - Client invokes: `shoppingCart.addItem(productId, quantity)`
3. **Proxy Action:**
   - Proxy converts method call to network message
   - Sends marshalled request across network
4. **Server Side:**
   - Skeleton receives the network request
   - Unmarshals it back to method call
   - Invokes actual `ShoppingCart.addItem()` on server object
5. **Response:**
   - Server object updates its state (adds item to cart)
   - Skeleton marshals response and sends back
   - Proxy receives response and returns to client

Another Example: Video Streaming Service

Client Object: `VideoPlayer`

- **State:** current position, volume, quality settings
- **Methods:** `play()`, `pause()`, `seek(position)`, `setQuality()`

Server Object: `VideoStream`

- **State:** video file data, user permissions, bandwidth info
- **Methods:** `getVideoChunk()`, `validateAccess()`, `adjustQuality()`

Flow:

1. Client calls `videoPlayer.play()`
2. Proxy marshals request → Network → Skeleton
3. Server's `VideoStream.getVideoChunk()` is invoked
4. Video data is sent back through skeleton → Network → Proxy
5. Client receives and displays video

Key Benefits

Transparency

Clients interact with remote objects exactly like local objects. The complexity of network communication is hidden.

Reusability

Same server objects can serve multiple different clients (web, mobile, desktop apps).

Scalability

Objects can be distributed across multiple servers based on load.

Maintainability

Business logic is centralized on servers, making updates easier.

Common Technologies

- **Java RMI (Remote Method Invocation)**
- **CORBA (Common Object Request Broker Architecture)**
- **Microsoft .NET Remoting**
- **gRPC** (modern implementation)

Summary

Object-based architecture allows you to build distributed applications where objects on different machines can communicate seamlessly. The proxy-skeleton pattern handles all the networking complexity, making remote method calls appear just like local method calls to developers.